

Esame OS Internals (Cabodi)

Data: 05/07/2021

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Inverted Page Table e Dimensionamento)

Contesto:

A) Si considerino le affermazioni che seguono, a proposito di possibili vantaggi e svantaggi di una inverted page table (IPT), rispetto a una tabella delle pagine standard (PT). Si dica di ognuna se sia vera o falsa, motivando la risposta:

- **Vantaggi:** L'IPT permette di risparmiare memoria:

1. Si risparmia sempre memoria.
2. Dipende dalle dimensioni della RAM, dal numero di processi e dal loro spazio di indirizzamento virtuale.
3. Si risparmia sempre memoria quando lo spazio di indirizzamento di un processo è maggiore della dimensione della RAM.
4. Si può risparmiare anche in casi in cui lo spazio di indirizzamento di ogni processo è inferiore alla dimensione della RAM.

- **Svantaggi:** L'IPT è lenta, perché non garantisce accesso diretto ma occorre una ricerca:

1. La chiave di ricerca è il frame.
2. La chiave di ricerca è la pagina.
3. La chiave di ricerca è la coppia (pid, frame).
4. La chiave di ricerca è la coppia (pid, pagina).
5. Per migliorare le prestazioni si sostituisce la IPT con una tabella di HASH.
6. Per migliorare le prestazioni si aggiunge alla IPT una tabella di HASH.

B) Calcoli di memoria: Sia dato un processo avente spazio di indirizzamento virtuale di 48 GB, in un sistema dotato di 16GB di RAM, con architettura a 64 bit (in cui si indirizza il Byte) e gestione della memoria paginata (pagine/frame da 4KB) per i soli processi user, quindi non per il kernel. Si supponga che 4GB di RAM siano allocati in modo statico al kernel. Si vogliono confrontare una soluzione basata su tabella delle pagine standard (una tabella per ogni processo) e una basata su IPT. Si calcolino:

B1) Le dimensioni della PT (a un solo livello) per il processo e della IPT. Si ipotizzi che il pid di un processo possa essere rappresentato su 12 bit. Si utilizzino 28 bit per gli indici di pagina e/o di frame (nella PT o nella IPT) e si tenga conto che, per allineamento, una cella di IPT o PT può solo essere di 32 o 64 bit.

B2) Si dica infine, utilizzando la IPT proposta (12 bit di pid, 28 bit per un indice di pagina/frame), quale è la dimensione massima possibile per lo spazio di indirizzamento virtuale di un processo.

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF: Mean Time To Failure), il tempo medio per riparazione (MTTR: Mean Time To Repair), e il tempo medio tra perdite di dato (MTTDL: Mean Time To Data Loss).

Si risponda alle seguenti domande:

-) E' possibile che (motivare le risposte):
 - A1) $MTTR > MTTF$?
 - A2) $MTTDL > 10 \cdot MTTF$?
-) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTF = 20000$ ore, quanto deve essere MTTR per garantire $MTTDL > 100 \cdot MTTF$?

Domanda 3 (I/O, DMA e Cycle Stealing)

Contesto: Si consideri il problema della gestione di una richiesta di I/O a blocchi realizzato mediante DMA.

Si risponda alle seguenti domande:

-) Che cosa si intende, in questo contesto, con il termine "cycle stealing"?
-) Perché il trasferimento in DMA è vantaggioso rispetto all'I/O programmato?
-) Immaginando di dover trasferire 10MB di dati da disco a memoria RAM, quanti Byte transitano sul bus dati della RAM nei due casi (trasferimento in DMA e I/O programmato)? (Motivare).
-) Qualora si volesse usare per l'I/O, mediante DMA, un buffer in memoria Kernel, invece di usare direttamente la sorgente/destinazione in memoria user, il DMA controller farebbe più o meno trasferimenti, oppure lo stesso numero, rispetto alla versione senza buffer? (Motivare).

SECTION 2: OS161

Domanda 4 (Copia di un Address Space in OS161)

Contesto: Sia dato un sistema OS161. Si consideri la funzione `as_copy`, avente come obiettivo quello di "duplicare" un address space, cioè creare un nuovo address space a partire da uno esistente di cui si fa una copia. La `as_copy` potrebbe essere usata, ad esempio, per realizzare la system call `fork`, che genera un processo child a partire da un processo (parent) esistente. Si propone una possibile implementazione della `as_copy`:

```
int as_copy(struct addrspace *old, struct addrspace **ret) {
    struct addrspace *new;
    dumbvm_can_sleep();
    new = as_create();

    new->as_vbase1 = old->as_vbase1;
    new->as_npages1 = old->as_npages1;
    new->as_vbase2 = old->as_vbase2;
    new->as_npages2 = old->as_npages2;
    new->as_pbase1 = old->as_pbase1;
    new->as_pbase2 = old->as_pbase2;
    new->as_stackpbase = old->as_stackpbase;

    memmove((void *)PADDR_TO_KVADDR(new->as_pbase1),
            (const void *)PADDR_TO_KVADDR(old->as_pbase1),
            old->as_npages1 * PAGE_SIZE);

    memmove((void *)PADDR_TO_KVADDR(new->as_pbase2),
            (const void *)PADDR_TO_KVADDR(old->as_pbase2),
            old->as_npages2 * PAGE_SIZE);

    memmove((void *)PADDR_TO_KVADDR(new->as_stackpbase),
            (const void *)PADDR_TO_KVADDR(old->as_stackpbase),
            DUMBVM_STACKPAGES * PAGE_SIZE);

    *ret = new;
    return 0;
}
```

Si risponda alle seguenti domande:

-) Perché il parametro `old` ha un solo asterisco mentre `ret` ne ha due?
-) La funzione `as_create` alloca RAM per i due segmenti (testo e dati) e per lo stack del nuovo address space?
-) La funzione contiene volutamente degli errori. Si tratta di errori semantici, legati all'obiettivo della funzione, quindi non di errori sintattici: si individuino tali errori e si spieghi perché lo sono.
-) La funzione `memmove` viene qui utilizzata per copiare il contenuto di una parte di RAM fisica in un'altra parte di RAM fisica: che cosa realizza `PADDR_TO_KVADDR` e perché viene utilizzata in questo contesto, invece di chiamare semplicemente, ad esempio: `memmove((void*)(new->as_pbase1), (const void*)(old->as_pbase1), old->as_npages1*PAGE_SIZE);`?

Domanda 5 (Implementazione System Call `sys_exit`)

Contesto: In un sistema OS161, si vuole realizzare il supporto per la system call `SYS_exit` (che corrisponde alla `void exit(int status);`). NON SI INTENDE ANCORA SUPPORTARE LA `waitpid`, ma unicamente il salvataggio dello stato (`status`), la terminazione del thread e il rilascio dell'address space.

Si risponda alle seguenti domande:

-) Si supponga che la funzione `syscall()`, in corrispondenza al valore `SYS_exit` in `callno` chiami la funzione `my_sys_exit()`, si definisca il prototipo di tale funzione e se ne scriva la chiamata (con parametri attuali) in `syscall()`.

```
/* METTERE TIPO DI RITORNO */
my_sys_exit(/* METTERE PARAMETRI QUI */);

void syscall(struct trapframe *tf) {
    /* ... */
    callno = tf->tf_v0;
    /* ... */
    switch (callno) {
        /* ... */
        case SYS_exit:
            err = my_sys_exit(/* METTERE PARAMETRI QUI */);
            break;
    }
}
```

-) Si osservi il seguente frammento di codice (incompleto) della `my_sys_exit()`. Si dica da dove può o deve provenire il valore `status`, e a che variabile o campo di struct va assegnato (dire quindi cosa va sostituito ai ...). Si identifichino e si commentino gli errori o problemi contenuti nelle successive istruzioni:

```
my_sys_exit (...) {
    ... = status;
    thread_exit(curthread);
    proc_destroy(curproc);
    as_destroy(curproc->p_addrspace);
}
```

ANSWERS

Domanda 1 (Inverted Page Table e Dimensionamento)

A) Vantaggi e Svantaggi dell'IPT rispetto alla PT (Vero/Falso con motivazioni):

Vantaggi (L'IPT permette di risparmiare memoria):

- **1. FALSO.** Non si risparmia *sempre*. Se il sistema ha una grande quantità di memoria fisica (RAM) ma ospita pochissimi processi con uno Spazio di Indirizzamento Virtuale (VAS) molto ridotto, la somma delle dimensioni delle PT convenzionali risulterà inferiore all'ingombro dell'IPT (che scala con la RAM).
- **2. VERO.** La dimensione totale occupata dalle Page Table standard dipende dal VAS di ogni processo moltiplicato per il numero di processi attivi. Al contrario, la dimensione dell'IPT è strettamente determinata dalla dimensione della RAM fisica divisa per la dimensione della pagina.
- **3. FALSO.** Il termine "sempre" rende l'affermazione errata. Se il VAS supera la RAM di una percentuale irrisoria (es. $VAS = 1.1 \times RAM$), ma l'allineamento dei campi rende l'entry dell'IPT grande il doppio rispetto all'entry della PT (es. 8 Byte per IPT contro 4 Byte per PT), la tabella PT per quel singolo processo occuperà comunque meno memoria dell'IPT di sistema.
- **4. VERO.** È il caso d'uso tipico: anche se ogni singolo processo ha un VAS minore della RAM, l'esecuzione concorrente di decine o centinaia di processi farà sì che la somma della memoria occupata da tutte le PT convenzionali ecceda di gran lunga la singola e centralizzata IPT.

Svantaggi (L'IPT è lenta):

- **1, 2, 3. FALSO.** (Vedi motivazione al punto 4).
- **4. VERO.** A differenza della PT che usa il Virtual Page Number come indice diretto di un array, l'IPT ha come indice il frame fisico. Quando la MMU riceve un indirizzo logico, deve cercare all'interno della tabella quale frame possiede la combinazione univoca (**PID, numero di pagina**). La chiave di ricerca è quindi tale tupla.
- **5. FALSO.** La tabella di Hash non sostituisce l'IPT, in quanto l'IPT è necessaria per contenere l'effettivo mapping con i frame fisici limitati dalla RAM.
- **6. VERO.** A causa della ricerca tramite chiave (PID, pagina), la scansione lineare dell'IPT richiederebbe tempo $O(N)$. Si *aggiunge* quindi una tabella di Hash che, prendendo in input la chiave, punti rapidamente all'entry corretta dell'IPT, abbattendo i tempi a $O(1)$.

B) Calcoli di memoria (PT vs IPT):

B1) Dimensioni della PT e della IPT:

Dati di base:

- $VAS = 48 \text{ GB} = 48 \times 2^{30} \text{ Byte}$
- $\text{RAM utente} = 16 \text{ GB} - 4 \text{ GB (kernel)} = 12 \text{ GB} = 12 \times 2^{30} \text{ Byte}$
- Dimensione pagina/frame = $4 \text{ KB} = 2^{12} \text{ Byte}$

Calcolo PT (singolo processo):

- Numero di entries = $\frac{VAS}{\text{Dimensione pagina}} = \frac{48 \times 2^{30}}{2^{12}} = 48 \times 2^{18} = 12 \times 2^{20}$ entries (circa 12 Milioni).
- Dimensione entry: L'indice di frame è di 28 bit. Per allineamento, la cella deve occupare $32 \text{ bit} = 4 \text{ Byte}$.
- Dimensione totale PT = $12 \times 2^{20} \times 4 \text{ B} = 48 \times 2^{20} \text{ B} = 48 \text{ MB}$.

Calcolo IPT (sistema globale):

- Numero di entries = $\frac{\text{RAM utente}}{\text{Dimensione frame}} = \frac{12 \times 2^{30}}{2^{12}} = 3 \times 2^{20}$ entries (circa 3 Milioni).
- Dimensione entry: Necessita del PID (12 bit) + indice di pagina (28 bit) = 40 bit. Per allineamento, la cella deve necessariamente occupare $64 \text{ bit} = 8 \text{ Byte}$.
- Dimensione totale IPT = $3 \times 2^{20} \times 8 \text{ B} = 24 \times 2^{20} \text{ B} = 24 \text{ MB}$.

B2) Dimensione massima del VAS con questa IPT:

L'architettura della IPT stabilisce che l'indice di pagina sia codificato su 28 bit. Sapendo che la dimensione della pagina è 4 KB, l'offset all'interno della pagina richiede $\log_2(4096) = 12 \text{ bit}$. Il numero totale di bit che compongono un indirizzo virtuale valido è la somma dei bit di pagina e dei bit di offset:

$$\text{Bit totali} = 28 + 12 = 40 \text{ bit.}$$

Di conseguenza, la dimensione massima possibile per lo Spazio di Indirizzamento Virtuale di un singolo processo è:

$$VAS_{\max} = 2^{40} \text{ Byte} = 1 \text{ TB.}$$

Domanda 2 (RAID e Affidabilità)

A) Valutazione delle casistiche di affidabilità:

- **A1) È possibile che $MTTR > MTTF$? SÌ (teoricamente).**

Motivazione: Matematicamente è possibile definire un sistema con questi parametri, ma ingegneristicamente rappresenta un sistema disfunzionale. Se il tempo medio di riparazione (*Mean Time To Repair*) supera il tempo medio di vita di un singolo disco (*Mean Time To Failure*), significa che i dischi si guastano più velocemente di quanto il sistema riesca a sostituirli o ricostruirli. In tale scenario, l'MTTDL collassa e la perdita di dati è praticamente garantita a stretto giro.

- **A2) È possibile che $MTTDL > 10 \times MTTF$? SÌ.**

Motivazione: È non solo possibile, ma è lo scopo primario dell'invenzione dei sistemi RAID. Introducendo ridondanza (mirroring o parità), il sistema è in grado di sopravvivere al guasto di singoli componenti. Nelle formule di affidabilità dei RAID tolleranti ai guasti (come RAID 1 o RAID 5), il tempo medio alla perdita dei dati (MTTDL) cresce proporzionalmente al quadrato (o al cubo, nel RAID 6) dell'MTTF, permettendo al sistema di raggiungere tempi di vita attesi di ordini di grandezza nettamente superiori (spesso migliaia o milioni di volte) rispetto al singolo disco.

B) Calcolo del MTTR vincolato su struttura RAID 1:

Dati iniziali:

- Configurazione: RAID 1 (Mirrored) con 2 dischi (1 primario + 1 replica).
- $MTTF = 20.000$ ore.
- Vincolo richiesto: $MTTDL > 100 \times MTTF$.

Il modello teorico per il calcolo del MTTDL di un array RAID 1 a 2 dischi, assumendo guasti indipendenti, è dato da:

$$MTTDL = \frac{MTTF^2}{2 \times MTTR}$$

(Ottenuto dividendo il tempo atteso per il primo guasto, $MTTF/2$, per la probabilità di guasto fatale del disco superstite durante la riparazione, $MTTR/MTTF$).

Imponiamo il vincolo richiesto dalla traccia:

$$\frac{MTTF^2}{2 \times MTTR} > 100 \times MTTF$$

Dividiamo entrambi i membri per $MTTF$ (quantità strettamente positiva):

$$\frac{MTTF}{2 \times MTTR} > 100$$

Moltiplichiamo per $2 \times MTTR$ (assumendo $MTTR > 0$) e dividiamo per 100:

$$MTTR < \frac{MTTF}{200}$$

Sostituiamo il valore numerico del $MTTF$ fornito dalla traccia:

$$MTTR < \frac{20.000}{200}$$

$$MTTR < \mathbf{100 \text{ ore}}$$

Affinché il sistema RAID 1 garantisca il livello di affidabilità richiesto, le operazioni di rilevamento del guasto, sostituzione dell'unità e ricostruzione dei dati devono essere completate tassativamente in **meno di 100 ore** medie.

Domanda 3 (I/O, DMA e Cycle Stealing)

A) Significato di "Cycle Stealing":

Il "Cycle Stealing" (furto di cicli) è il fenomeno per cui il controllore DMA, per poter eseguire il trasferimento dei dati in memoria principale, deve acquisire il ruolo di *bus master* sottraendo temporaneamente il controllo del bus di sistema alla CPU. Poiché la CPU e il DMA condividono lo stesso bus dati/indirizzi, quando il DMA effettua un trasferimento la CPU viene "messa in pausa" per alcuni cicli di clock qualora tenti di accedere alla memoria, subendo di fatto un "furto" di cicli utili per le proprie operazioni.

B) Vantaggio del trasferimento in DMA:

Il vantaggio fondamentale del DMA rispetto all'I/O programmato è l'**offloading della CPU**. Nell'I/O programmato, la CPU è costretta a gestire personalmente il trasferimento byte per byte (o word per word), sprecando tempo prezioso. Con il DMA, la CPU si limita a configurare i registri del controllore (indirizzo sorgente, indirizzo destinazione, dimensione) e poi può riprendere l'esecuzione di altre istruzioni (o di altri processi). Il DMA lavora in completa autonomia (*background*) e notifica la CPU unicamente al termine dell'intero blocco di dati tramite un singolo *Interrupt*, massimizzando la concorrenza e l'efficienza del sistema.

C) Transito dei dati sul bus (10 MB):

I due meccanismi generano un traffico sul bus drasticamente differente:

- **Trasferimento in DMA $\Rightarrow$ 10 MB transitati.** Il controllore DMA sposta i dati direttamente dal buffer del dispositivo hardware alla RAM (*Direct Memory Access*). Il blocco da 10 MB viaggia sul bus una singola volta.
- **I/O Programmato (PIO) $\Rightarrow$ 20 MB transitati.** La CPU fa da intermediario obbligatorio. Per spostare ogni singolo dato, la CPU deve prima eseguire una lettura (LOAD) dal controller del disco verso i propri registri interni (facendo transitare i 10 MB dal dispositivo alla CPU) e successivamente eseguire una scrittura (STORE) verso la RAM (facendo transitare nuovamente gli stessi 10 MB dalla CPU alla memoria). Il dato percorre il bus di sistema due volte.

D) Utilizzo di un buffer Kernel per l'I/O con DMA:

Il controllore DMA eseguirebbe lo stesso identico numero di trasferimenti.

Motivazione: Il compito del DMA è unicamente quello di trasferire i dati dalla periferica alla memoria (o viceversa). Che l'indirizzo di destinazione fornitogli dalla CPU appartenga allo spazio utente o a un buffer interno del kernel, per il DMA l'operazione è fisicamente identica: trasferirà esattamente i byte richiesti. L'onere aggiuntivo causato dall'utilizzo del buffer nel kernel è il successivo trasferimento *memory-to-memory* (dal kernel buffer all'user buffer) per la consegna definitiva dei dati, ma questa copia è a carico esclusivo della CPU (tramite istruzioni come *copyout*), non del controllore DMA.

Domanda 4 (Copia di un Address Space in OS161)

A) Differenza tra i parametri `old (*)` e `ret (**)`:

La differenza deriva dai vincoli del passaggio dei parametri in linguaggio C.

- `old` è un puntatore semplice (`struct addrspace *`). Viene passato per *valore* affinché la funzione possa leggere in sola lettura le informazioni contenute nell'Address Space originale del processo padre.
- `ret` è un puntatore a puntatore (`struct addrspace **`). Poiché il valore di ritorno formale della funzione è un intero utilizzato per restituire codici di errore (es. `return 0;` per il successo), si utilizza `ret` come parametro di *output*. Deriferenziandolo (`*ret = new;`), la funzione è in grado di modificare la variabile locale del chiamante, facendola puntare alla nuova struttura dati appena creata in memoria.

B) Allocazione RAM da parte di `as_create()`:

NO. La funzione `as_create()` **non** alloca le pagine di RAM fisica per i segmenti testo, dati o per lo stack. Il suo unico scopo è allocare lo spazio per la struttura dati contenitore (i *metadati*), ovvero la `struct addrspace` in sé, tipicamente attraverso una chiamata a `kmalloc`. L'effettiva allocazione dei frame fisici in RAM per i segmenti del processo (tramite funzioni come `getppages`) avviene in fasi successive (ad esempio in `as_prepare_load()`).

C) Errori semantici dell'implementazione proposta:

L'errore semantico è critico e riguarda il fallimento nell'isolamento della memoria tra processi.

- **Mancata allocazione fisica:** Il codice esegue assegnamenti diretti come `new->as_pbase1 = old->as_pbase1;`. In questo modo, l'Address Space del processo figlio viene configurato per puntare agli **stessi identici frame fisici** del processo padre.
- **Copia inutile:** A causa del problema precedente, la successiva chiamata a `memmove` utilizza lo stesso indirizzo fisico sia come destinazione che come sorgente. Il sistema sta copiando inutilmente i dati su loro stessi.

- **Violazione dell'isolamento:** Se questa `as_copy` fosse usata per una `fork`, `parent` e `child` condividerebbero interamente la stessa RAM (senza meccanismi *Copy-On-Write*), distruggendo l'indipendenza dei processi. La correzione richiederebbe l'allocazione esplicita di nuovi `pbase` per `new` prima di invocare la `memmove`.

D) Utilizzo e scopo della macro `PADDR_TO_KVADDR`:

I campi come `as_pbase1` memorizzano **indirizzi fisici** puri. Tuttavia, l'architettura della CPU (anche quando esegue istruzioni in Kernel Mode) lavora ed esegue letture/scritture esclusivamente passando per **indirizzi virtuali**.

Se si passassero i campi `pbase` direttamente a `memmove`, il processore (MMU) interpreterebbe quegli indirizzi fisici come se fossero indirizzi virtuali, portando a errori di accesso (*TLB Miss* o *Address Error*). La macro `PADDR_TO_KVADDR` prende l'indirizzo fisico in ingresso e lo converte in un indirizzo virtuale del kernel (mappato nel segmento speciale `kseg0`), garantendo che le operazioni di `memmove` operino sulle corrette celle di memoria fisica in modo sicuro.

Domanda 5 (Implementazione System Call `sys_exit`)

A) Prototipo e chiamata all'interno del dispatcher `syscall`:

La system call `exit` prende un singolo argomento intero (`status`). In OS161 (architettura MIPS), il primo argomento passato da *user space* viene salvato nel registro `a0`, accessibile nel kernel tramite `tf->tf_a0`.

Poiché lo `switch` si aspetta di assegnare un valore alla variabile intera `err`, il prototipo dovrà restituire un `int`.

```
/* PROTOTIPO */
int my_sys_exit(int status);

void syscall(struct trapframe *tf) {
    /* ... */
    callno = tf->tf_v0;
    /* ... */
    switch (callno) {
        /* ... */
        case SYS_exit:
            /* Il primo argomento è in a0 */
            err = my_sys_exit(tf->tf_a0);
            break;
    }
}
```

B) Gestione dello stato ed errori semantici:

Assegnamento dello status:

Il parametro `status` deve provenire dalla chiamata della funzione (e originariamente dal registro hardware). Affinché il valore sopravviva alla terminazione del thread e possa essere recuperato successivamente dal processo padre (tramite `waitpid`), esso deve essere memorizzato all'interno della struttura dati persistente del processo. L'istruzione va sostituita con:

```
curproc->p_exitstatus = status;
```

Identificazione degli errori nelle istruzioni successive:

1. **Codice irraggiungibile (Ordine di esecuzione errato):** La funzione `thread_exit()` termina irreversibilmente l'esecuzione del thread corrente, innescando lo scheduler e un

context switch senza ritorno. Pertanto, le funzioni `proc_destroy` e `as_destroy` poste *successivamente* non verranno **mai** eseguite. L'eliminazione dell'Address Space (`as_destroy`) deve obbligatoriamente precedere l'uscita del thread.

2. **Distruzione prematura del processo (Problema Architettureale):** La chiamata a `proc_destroy(curproc)` da parte del processo stesso è logicamente errata. Se il processo figlio distruggesse la propria `struct proc`, l'informazione `status` appena salvata andrebbe perduta (deallocata). Quando in futuro il padre invocherà la `waitpid`, proverebbe ad accedere a un'area di memoria liberata (*read-after-free*) o non troverebbe il figlio. Il processo figlio deve terminare diventando uno "zombie", lasciando che sia il padre a invocare `proc_destroy` una volta consumato lo stato di uscita.
3. **Errore di API (thread_exit):** In OS161, la funzione `thread_exit` non prende parametri, poiché distrugge implicitamente il thread chiamante. L'istruzione `thread_exit(curthread)` è sintatticamente scorretta; dovrebbe essere semplicemente `thread_exit()`.